

Water Rationing System - Project TLDR

A concise explanation of what the project does, why each component exists, and how the current prototype works.

1. What Problem Are We Solving?

During heatwaves or water shortages, a society or hostel can run out of water unpredictably. Management usually has to guess how much water is left, when to ration it, and whether a pipeline leak is wasting supply.

This project turns that guesswork into a monitored, automated system. It measures tank level and water flow, predicts how long the water will last, detects abnormal flow loss, and can trigger rationing through an electronic valve.

2. The Big Idea

- Sensors measure tank level and flow rate in real time.
- ESP32 publishes sensor readings over MQTT.
- FastAPI backend receives telemetry, calculates status, predicts time-to-depletion, and sends valve commands.
- InfluxDB stores telemetry as time-series data.
- Grafana visualizes tank level, flow rate, time-to-depletion, volume loss, and valve actions.
- A solenoid valve can physically restrict supply when shortage or leak conditions are critical.

3. What Each Part Does

Component	Purpose
ESP32	Reads physical sensors and controls the solenoid valve. In the current software demo, simulator data stands in for ESP32 readings.
HC-SR04 Ultrasonic Sensor	Measures distance from tank top to water surface, then converts that into tank fill percentage.
YF-S201 Flow Sensor	Measures water flow in liters per minute using pulse counts. A second flow value can simulate downstream flow for leak detection.
Solenoid Valve	Electronic tap controlled by ESP32. It opens normally and restricts or closes when rationing is triggered.
MQTT / Mosquitto	Lightweight messaging layer between ESP32/simulator and backend. Telemetry goes in; valve commands come back out.
FastAPI Backend	Brain of the prototype: validates telemetry, predicts time-to-depletion, detects leaks, stores data, and publishes valve commands.
InfluxDB	Time-series database optimized for sensor readings and dashboard queries.
Grafana	Live dashboard for examiner/demo view: tank level, flow, predicted depletion, volume loss, and valve commands.
Docker Compose	Starts the local stack consistently: MQTT, InfluxDB, backend, and Grafana.

4. Current Prototype Status

- GitHub repository is public: [TrijalBurman/Water-Rationing-System](#).
- Work is happening on branch `trijal`.
- The local Docker stack runs Mosquitto, InfluxDB, FastAPI, and Grafana.
- The backend has forecast logic for time-to-depletion and anomaly logic for flow loss.
- Grafana dashboard is provisioned automatically and shows live demo data.
- A PowerShell demo script can publish normal, shortage, or leak scenarios.
- ESP32 firmware skeleton is present for real hardware integration.

5. Demo Scenarios

Scenario	What Happens	Expected Result
Normal	Tank level stays stable and upstream/downstream flow match.	System mode remains normal, valve stays open.
Shortage	Tank level drops quickly to simulate supply cut-off or heatwave demand.	Time-to-depletion falls below threshold, rationing mode starts, valve becomes restricted.
Leak	Downstream flow is lower than upstream flow.	Backend detects volume loss and raises a potential leak alert.

6. Daily Commands

Task	Command / URL
Start everything	<code>docker compose up --build -d</code>
Open Grafana	<code>http://127.0.0.1:3000</code> login: admin / admin
Open API docs	<code>http://127.0.0.1:8000/docs</code>
Run normal demo	<code>.\scripts\demo.ps1 -Scenario normal -Seconds 20 -SampleMinutes 1</code>
Run shortage demo	<code>.\scripts\demo.ps1 -Scenario shortage -Seconds 20 -SampleMinutes 1</code>
Run leak demo	<code>.\scripts\demo.ps1 -Scenario leak -Seconds 20 -SampleMinutes 1</code>
Stop everything	<code>docker compose down</code> ; then quit Docker Desktop and run <code>wsl --shutdown</code>

7. TLDR in One Sentence

We are building a demo-ready IoT water management system that senses water availability, predicts shortage risk, detects leaks, displays everything on a dashboard, and can automatically ration water through a valve.